

Практическая работа 20. Создайте арт-пространство

Применяя полученные в этом разделе знания, вы создадите собственное цифровое арт-пространство - приложение, в котором будет отображаться множество работ, которые вы можете продемонстрировать. В отличие от предыдущих работ, где вам давались пошаговые инструкции, здесь вам даются только рекомендации и предложения о том, что можно создать с помощью концепций, которые вы изучили до сих пор. Вам предлагается использовать свои творческие способности для самостоятельного создания приложения при ограниченном руководстве.

Самостоятельное создание приложения - сложная задача, но не волнуйтесь, ведь у вас было достаточно практики! Вы можете использовать те же навыки, которые вы изучили, в этом новом контексте. Вы всегда можете обратиться к предыдущим кодовым лабораториям, если не уверены в том, как реализовать определенные части приложения.

Когда вы самостоятельно создаете это приложение и решаете проблемы, возникающие на пути, вы учитесь быстрее и дольше сохраняете концепции. Кроме того, приложение полностью настраивается, поэтому вы можете использовать его для демонстрации своего творения в портфолио разработчика!

Внимание: Не используйте для своего приложения активы, которые вам не принадлежат или на которые у вас нет соответствующих прав.

Необходимые условия Умение создавать и запускать проект в Android Studio. Опыт работы с синтаксисом Kotlin, включающим булевы выражения и выражения when. Умение применять основные концепции Jetpack Compose, такие как использование состояния с помощью объекта `MutableState`. Опыт работы с композитными функциями, включая композитные функции `Text`, `Image` и `Button`. Что вы узнаете Как создавать прототипы низкой точности и воплощать их в коде. Как создавать простые макеты с помощью композитных элементов `Row` и `Column` и упорядочивать их с помощью параметров `horizontalAlignment` и `verticalArrangement`. Как настраивать элементы Compose с помощью объекта `Modifier`. Как определять состояния и изменять их при срабатывании триггеров, например нажатии кнопок. Что вы создадите Приложение для Android, которое может отображать произведения искусства или семейные фотографии. Что вам понадобится Компьютер с установленной программой Android Studio. Фотографии или цифровые произведения искусства для отображения в приложении. Вот пример того, как может выглядеть ваше приложение в конце этого проекта:

Пример приложения Art Space, отображающего коллекцию произведений искусства в рамке, разработанный командой Android Basics Training.

2. Создание статичного пользовательского интерфейса с помощью композитных элементов

Создание прототипа низкой точности Прототип низкой точности (low-fidelity) - это простая модель или рисунок, который дает базовое представление о том, как выглядит приложение.

Создайте прототип низкой точности:

Подумайте о том, что вы хотите отобразить в приложении Art Space и кто является целевой аудиторией. На предпочитаемом вами носителе добавьте элементы, из которых будет состоять ваше приложение. Некоторые элементы, которые следует рассмотреть, включают: Изображение произведения искусства Информация о произведении, например, его название, художник и год публикации. Любые другие элементы, например, кнопки, которые делают приложение интерактивным и динамичным. Разместите эти элементы в разных позициях, а затем оцените их визуально. Не беспокойтесь о том, что с первого раза все получится идеально. Вы всегда можете остановиться на одном дизайне сейчас и итеративно улучшать его в дальнейшем. Примечание: Существуют принципы, которые помогают сделать дизайн лучше для пользователей, что выходит за рамки данного проекта. Чтобы узнать больше, смотрите раздел «Понимание верстки».

Возможно, у вас получится дизайн, похожий на это изображение:

Figure 1. Placeholder elements in UI mockups help to visualize the end product.

Преобразование дизайна в код Используйте свой прототип, чтобы помочь воплотить дизайн в коде:

Определите элементы пользовательского интерфейса, необходимые для создания вашего приложения. Например, из примера дизайна, который вы сделали, следует, что в вашем коде должны присутствовать один компонент Image, два компонента Text и два компонента Button.

Определите различные логические секции приложения и нарисуйте границы вокруг них. Этот шаг поможет вам разделить экран на небольшие составные части и продумать иерархию составных частей.

В этом примере вы можете разделить экран на три секции:

Стена с произведениями искусства
Дескриптор художественного произведения
Контроллер отображения
Каждый из этих разделов можно расположить с помощью компоновочных элементов, например компоновочного элемента Row или Column.

Рисунок 2. Границы вокруг разделов помогают разработчикам концептуализировать составные элементы.

Для каждого раздела приложения, содержащего несколько элементов пользовательского интерфейса, нарисуйте границы вокруг них. Эти границы помогают you see how one element relates to another in the section.

Рисунок 3. Дополнительные границы вокруг текста и кнопок помогают разработчикам компоновать составные элементы.

Теперь легче понять, как можно расположить составные элементы, такие как текст и кнопки, с помощью составных элементов макета.

Некоторые заметки о различных композитных элементах, которые вы можете использовать:

Композитные элементы Row или Column. Поэкспериментируйте с различными параметрами horizontalArrangement и verticalAlignment в композитах Row и Column, чтобы они соответствовали вашему дизайну. Составные изображения. Не забудьте заполнить параметр contentDescription. Как уже

говорилось в предыдущем коделабе, TalkBack использует параметр `contentDescription` для обеспечения доступности приложения. Если композит `Image` используется только в декоративных целях или есть элемент `Text`, который описывает композит `Image`, вы можете установить параметр `contentDescription` равным `null`.

Составные элементы текста. Вы можете экспериментировать с различными значениями `fontSize`, `textAlign` и `fontWeight`, чтобы придать тексту стиль. Вы также можете использовать функцию `buildAnnotatedString` для применения нескольких стилей к одному текстовому композиту. Составные элементы поверхности. Вы можете экспериментировать с различными значениями `Elevation`, `Color` и `BorderStroke` для `Modifier.border`, чтобы создавать различные пользовательские интерфейсы в композитных материалах `Surface`. Интервалы и выравнивание. Вы можете использовать аргументы модификатора, такие как `padding` и `weight`, чтобы помочь с расположением композитов. Примечание: Для простого приложения вы можете стилизовать каждый элемент пользовательского интерфейса самостоятельно. Однако такой подход приводит к накладным расходам по мере добавления новых экранов. Compose помогает поддерживать согласованность дизайна благодаря реализации `Material Design`. Подробнее о `Material Design` и `Material Theming` вы узнаете в следующих разделах. Чтобы узнать больше, смотрите раздел `Material Theming` в Compose.

Запустите приложение в эмуляторе или на устройстве Android.

Рисунок 4. Это приложение показывает статичный контент, но пользователи пока не могут с ним взаимодействовать.

3. Сделайте приложение интерактивным

Определите взаимодействие с пользователем Преимущество создания арт-пространства в цифровом формате заключается в том, что вы можете сделать его интерактивным и динамичным для ваших пользователей. В первоначальном проекте вы создали две кнопки для взаимодействия с пользователями. Однако это ваше собственное арт-пространство! Вы можете менять дизайн и способы взаимодействия пользователей с приложением по своему усмотрению. Теперь уделите минутку и подумайте, как вы хотите, чтобы пользователи взаимодействовали с приложением и как приложение должно реагировать на эти взаимодействия. Некоторые возможные варианты взаимодействия, которые вы можете добавить в свое приложение, включают:

Показ следующего или предыдущего произведения при нажатии кнопки. Перемотка вперед к следующему альбому при пролистывании. Показывать всплывающую подсказку для дополнительной информации при длительном нажатии кнопки. Примечание: Compose поддерживает множество жестов и анимаций, чтобы сделать ваше приложение интерактивным. Подробнее об анимации вы узнаете в следующих разделах. Дополнительные сведения о продвинутых темах см. в разделе Жесты.

Создание состояний для динамических элементов Поработайте над той частью пользовательского интерфейса, которая показывает следующую или предыдущую иллюстрацию при нажатии кнопки:

Сначала определите элементы пользовательского интерфейса, которые должны меняться при взаимодействии с пользователем. В данном случае элементами пользовательского интерфейса являются изображение произведения искусства, название произведения, исполнитель и год.

При необходимости создайте состояние для каждого из динамических элементов пользовательского интерфейса с помощью объекта `MutableState`. Не забудьте заменить жестко закодированные значения определенными состояниями. Примечание: Хотя сейчас вы используете одно состояние для каждого динамического элемента пользовательского интерфейса, такой подход может оказаться не самым эффективным с точки зрения читаемости кода и производительности приложения. Потенциально вы можете сгруппировать связанные элементы в одну сущность и объявить ее как одно состояние, что вы научитесь делать с помощью классов `Collection` и `Data` в следующих разделах. После изучения этих концепций вы сможете вернуться к этому проекту и рефакторить свой код с учетом изученных концепций.

Написание условной логики для взаимодействия Подумайте, какое поведение вам нужно, когда пользователь нажимает на кнопки, начиная с кнопки `Next`. Когда пользователь нажимает кнопку `Next`, он должен ожидать увидеть следующее произведение искусства в последовательности. На данный момент может быть сложно определить, какое произведение будет отображаться следующим.

Добавьте идентификаторы, или `ID`, в виде последовательных номеров, начинающихся с 1, для каждого произведения искусства. Теперь понятно, что следующее произведение искусства относится к тому произведению, которое имеет следующий `ID` в последовательности.

Поскольку у вас не бесконечное количество произведений искусства, вам также может понадобиться определить поведение кнопки «Далее» при отображении последнего произведения искусства в серии. Обычным поведением является возврат к отображению первого произведения искусства после последнего произведения искусства.

Сначала пишите в псевдокоде, чтобы передать логику кода без синтаксиса `Kotlin`. Если нужно показать три произведения искусства, псевдокод для логики кнопки `Next` может выглядеть примерно так:

```
if (current artwork is the first artwork) {  
    // Update states to show the second artwork.  
}  
else if (current artwork is the second artwork) {  
    // Update states to show the third artwork.  
}  
else if (current artwork is the last artwork) {  
    // Update state to show the first artwork.  
}
```

Преобразуйте псевдокод в код на языке `Kotlin`. Вы можете использовать оператор `when` для построения условной логики вместо оператора `if else`, чтобы сделать код более читабельным, когда он управляет большим количеством примеров.

Чтобы эта логика выполнялась при нажатии на кнопку, поместите ее внутрь аргумента `Button` `composables onClick()`. Повторите те же шаги, чтобы построить логику для кнопки «Предыдущая». Запустите приложение и нажмите на кнопки, чтобы убедиться, что они переключают отображение на предыдущее или следующее произведение искусства.

4. Вызов: создание приложений для разных размеров экрана

Одна из сильных сторон Android - поддержка множества устройств и размеров экранов, а это значит, что созданное вами приложение может охватить широкий круг аудитории и использоваться множеством способов. Чтобы обеспечить наилучший опыт для всех пользователей, вы должны тестировать свои приложения на устройствах, на которых они будут работать. Например, в данном примере приложения вы, возможно, изначально разработали, создали и протестировали приложение для мобильных устройств в портретном режиме. Однако некоторые из ваших пользователей могут найти приложение удобным для использования на большом экране в ландшафтном режиме.

Хотя планшеты не являются основным поддерживаемым устройством для этого приложения, вы все равно хотите быть уверены, что приложение не сломается, если пользователи будут использовать его на большом экране.

Протестируйте свое приложение на планшете с большим экраном:

Если у вас нет планшетного устройства Android, создайте виртуальное устройство Android (AVD). Создайте и запустите приложение на планшетном AVD в ландшафтном режиме. Визуально проверьте, что ничего не выглядит неприемлемым, например, обрезанные элементы пользовательского интерфейса, неровное выравнивание или неработающие кнопки.

Рисунок 5. Приложение нуждается в доработке, чтобы корректно отображаться на больших устройствах.

Измените код, чтобы исправить все найденные ошибки. В качестве руководства обратитесь к базовым рекомендациям по совместимости приложений для больших экранов. Еще раз протестируйте приложение на планшете и телефоне, чтобы убедиться, что исправление ошибок работает на обоих типах устройств.

Рисунок 6. Теперь приложение хорошо выглядит на большом экране.

Примечание: Вы можете заметить, что многие приложения, поддерживающие планшеты и телефоны, могут выглядеть по-разному на экранах разных форм-факторов. Это происходит потому, что часто приложение поддерживает разные макеты для разных размеров экрана. Чтобы узнать больше, смотрите раздел Поддержка разных размеров экрана.